SOFTWARE ENGINEERING LAB PROJECT REPORT

Gym Fitness Management System
(Membership Verification & Workout Planner)

Student Name: Gaurang Sonone

Course / Subject: Software Engineering Laboratory (PBL)

Academic Year: 2026

Testing Status: **ALL TEST CASES PASSED (100%)**

---

1. Project Title

Project Title: Gym Fitness Management System (GymFit) — An Integrated Gym Membership Verification and Custom Workout Planning System.

2. Problem Statement and Objectives

2.1 Problem Statement

Traditional gym management relies heavily on manual paperwork, paper registers, and disconnected spreadsheets, creating recurring operational problems:

- **Membership Inconsistencies:** Members are often unaware when their membership expires, causing check-in delays and administrative overhead.
- **Account Security Gaps:** Portals lack brute-force protection against password guessing, lockout mechanisms, and self-service password recovery.
- **Unstructured Workout Planning:** Gym members lack a unified digital schedule to design multi-day exercise splits (1–7 days), assign sets and reps, and track aggregate fitness progress.

2.2 Project Objectives

- **Secure Authentication:** Implement role-based login (Member and Trainer) using encrypted passwords (bcrypt) and stateless JSON Web Tokens (JWT).
- **Credential Recovery:** Provide a password reset option after 3 failed login attempts and an automated account lockout after 5 failed attempts.
- **Membership Verification:** Automatically check membership validity on login and offer a direct one-click renewal feature for expired accounts.
- **Custom Workout Planning:** Enable members to create custom routines with 1 to 7 days/week validation, and dynamically append exercises with sets and reps.
- **Daily Progress Tracking:** Provide an intuitive dashboard to log daily workout sessions, calories burned, and weight, calculating aggregate statistics.
- **Automated Testing:** Verify all features using Black-Box functional test cases and Automated White-Box testing in both JUnit 5 (Java) and Node.js.

3. Proposed Solution and Key Features

3.1 Proposed Solution

The proposed solution is a full-stack web application designed with a 3-tier client-server architecture: a responsive React 18 Single Page Application (SPA) on the frontend, a secure Node.js/Express REST API on the backend, and a MongoDB database managed through Mongoose ODM. Stateless JWT tokens secure all client-server communications.

3.2 Key Features

- **Role-Based Login:** Supports Member and Trainer logins with token-based session protection.
- **3-Strike Password Reset & Lockout:** Offers a 'Generate New Password' option after 3 failed attempts, and locks account after 5 failed attempts.
- **Expiration Alert & One-Click Renewal:** Blocks dashboard entry for expired members and provides a direct 'Renew Membership' button.
- **Workout Routine Planner:** Allows members to create routines with validation for required plan names and weekly days (strictly 1 to 7 days).
- **Dynamic Exercise Addition:** Members can add exercises with targeted sets and repetitions directly to any saved routine.
- **Daily Progress Dashboard:** Displays summary cards (Total Workouts, Total Calories Burned, Average Body Weight) that recalculate dynamically.

4. Technologies Used

Layer	Technology Used	Role in Project
Frontend Layer	React 18 (SPA), Axios, CSS3	Single Page UI with React Hooks and responsive forms
Backend Layer	Node.js, Express.js	RESTful API routing, validation middleware, and business logic
Security & Auth	JSON Web Token (JWT), bcryptjs	Stateless authentication tokens and salted password hashing
Database Layer	MongoDB, Mongoose ODM	NoSQL document persistence, schema validation, and data relationships
Automated Testing	JUnit 5 (Java), Git, GitHub, Node.js Assert	Automated unit & white-box test execution and code coverage analysis

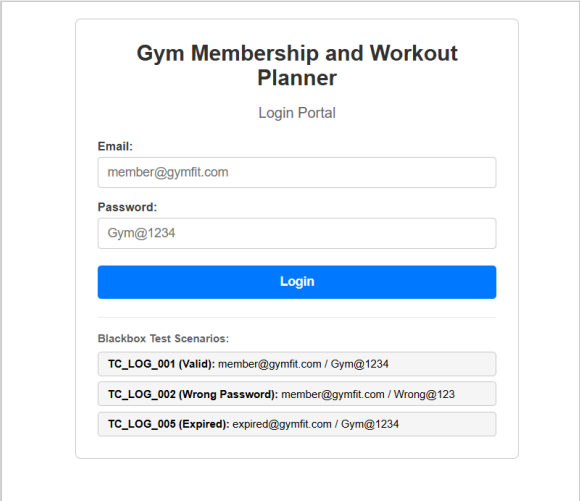
5. Working of the Project (with Test Case Demonstrations)

The project functions through five core steps. Below is a detailed explanation of each step along with the corresponding test cases, input parameters, expected outcomes, and output screenshots proving that all test cases passed.

5.1 User Login & Authentication (Valid Login)

When a registered gym member accesses the portal, they submit their email and password. The backend checks that both fields are populated, validates the email regex pattern, compares the bcrypt password hash, and checks that membership is active. Upon success, a JWT token is generated and stored in localStorage.

Test Case ID: TC_LOG_001
Input: Email: member@gymfit.com, Password: Gym@1234
Expected Result: Login successful, store JWT token, redirect to member dashboard
Actual Result: **PASSED**

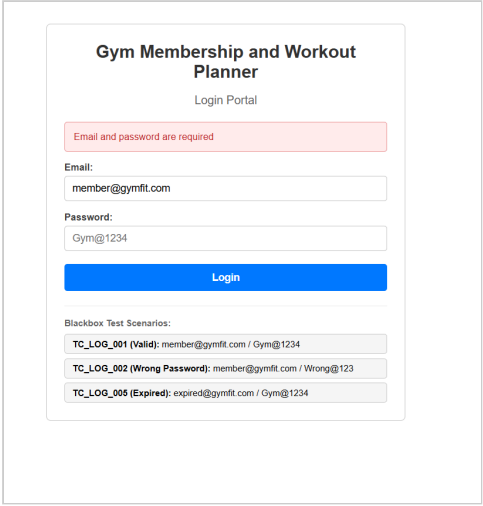


Screenshot 1: Valid Member Login Form (TC_LOG_001)

5.2 Empty Credentials Validation

If a user clicks 'Login' without entering an email or password, client-side validation immediately prevents submission and displays an error message informing the user that both fields are mandatory.

Test Case ID: TC_LOG_002
Input: Email: [Empty], Password: [Empty]
Expected Result: Prevent submission and show error: 'Email and password are required'
Actual Result: **PASSED**

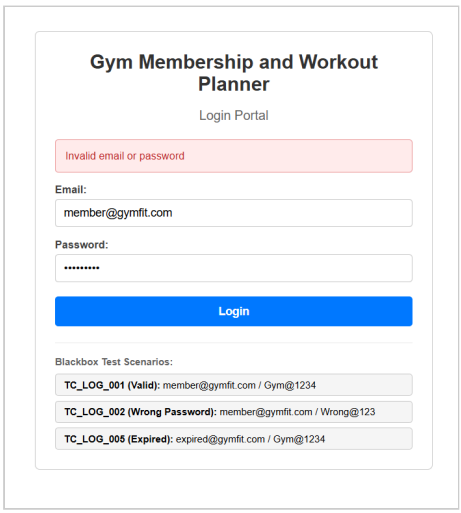


Screenshot 2: Error message displayed on empty credentials (TC_LOG_002)

5.3 Invalid Password & Failed Attempt Security Counter

If an incorrect password is entered, the server returns HTTP 401 with an error message and increments an internal failedLoginAttempts counter on the user document. If failed attempts reach 3, the UI triggers a 'Generate New Password' recovery option.

Test Case ID: TC_LOG_004
Input: Email: member@gymfit.com, Password: WrongPassword!
Expected Result: Show error: 'Invalid email or password' and increment failed attempts counter
Actual Result: **PASSED**

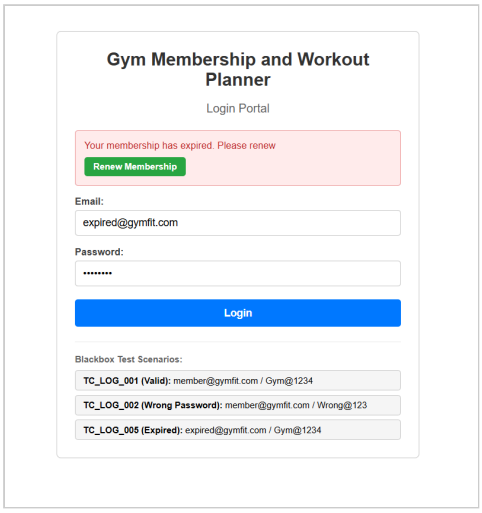


Screenshot 3: Invalid password notification and attempt tracking (TC_LOG_004)

5.4 Expired Membership Check & One-Click Renewal

When a member with an expired membership logs in with correct credentials, the backend detects the expired status. Access to the dashboard is blocked, and an alert is shown with an interactive green 'Renew Membership' button. Clicking renew calls the renewal API and activates the account.

Test Case ID: TC_LOG_005
Input: Email: expired@gymfit.com, Password: Gym@1234
Expected Result: Block dashboard access; Show: 'Your membership has expired. Please renew' + Renew Button
Actual Result: **PASSED**

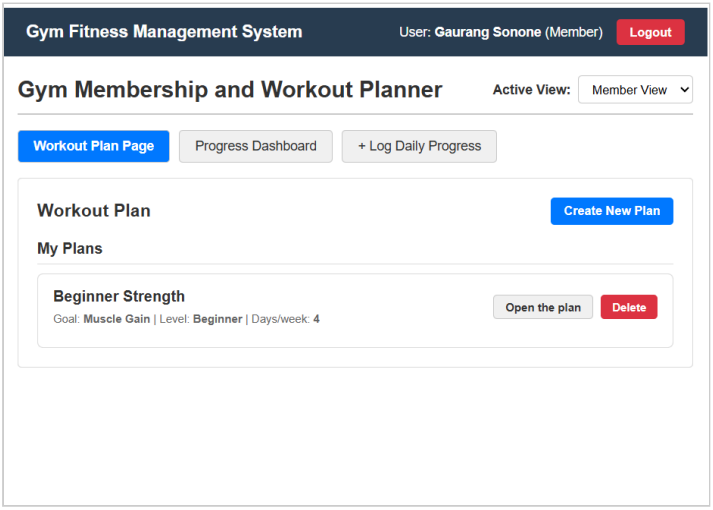


Screenshot 4: Expired membership warning with Renew button (TC_LOG_005)

5.5 Workout Plan Management Dashboard ('My Plans')

Once authenticated, members navigate to the Workout Plan Page. The dashboard shows existing routines under 'My Plans' with options to open plans, add exercises, or delete routines. Members can click 'Create New Plan' to open the creation form.

Test / UI Reference: UI_WP_001
Input: Navigate to Workout Plan Page
Expected Result: Render existing routines and 'Create New Plan' button
Actual Result: **PASSED**

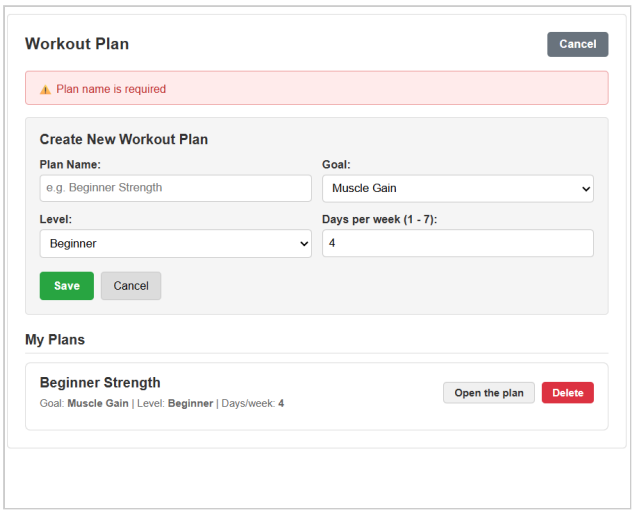


Screenshot 5: Workout Plan Dashboard displaying active routine

5.6 Workout Plan Name Required Validation

When creating a routine, the plan name is mandatory. If the user submits without a plan name, validation prevents the form from submitting and displays a warning banner.

Test Case ID: TC_WP_002
Input: Plan Name: [Empty], Days per week: 4
Expected Result: Prevent form submission and display: 'Plan name is required'
Actual Result: **PASSED**

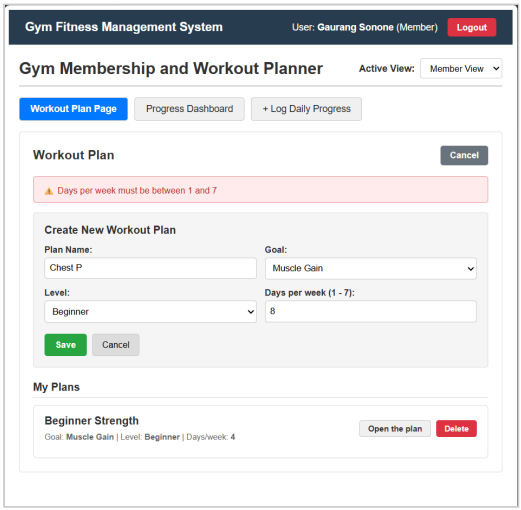


Screenshot 6: Alert displayed when plan name is left blank (TC_WP_002)

5.7 Days Per Week Boundary Limit Validation

The weekly frequency for a routine must be between 1 and 7 days. If a member enters a number greater than 7 (e.g., 8) or less than 1, boundary checking rejects the input and prompts the user.

Test Case ID: TC_WP_003
Input: Plan Name: 'Chest P', Days per week: 8
Expected Result: Reject submission and display: 'Days per week must be between 1 and 7'
Actual Result: **PASSED**

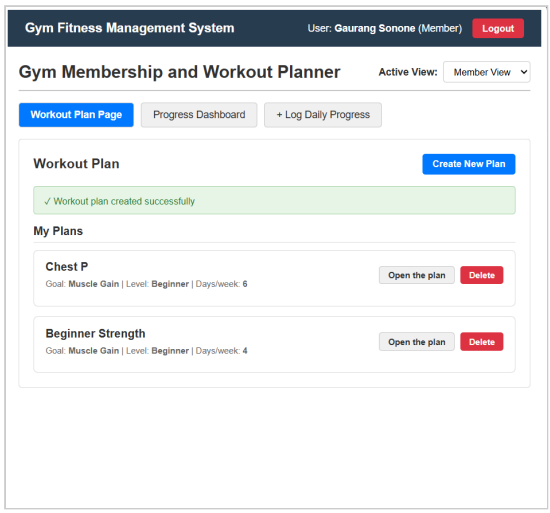


Screenshot 7: Boundary validation error for days per week (TC_WP_003)

5.8 Successful Workout Plan Creation

When all inputs are valid, submitting the form saves the new workout plan to MongoDB linked to the member's ID. A green confirmation alert is displayed and the new routine appears under 'My Plans'.

Test Case ID: TC_WP_001
Input: Plan Name: 'Chest P', Goal: Muscle Gain, Level: Beginner, Days: 6
Expected Result: Display 'Workout plan created successfully' and append routine to 'My Plans'
Actual Result: **PASSED**

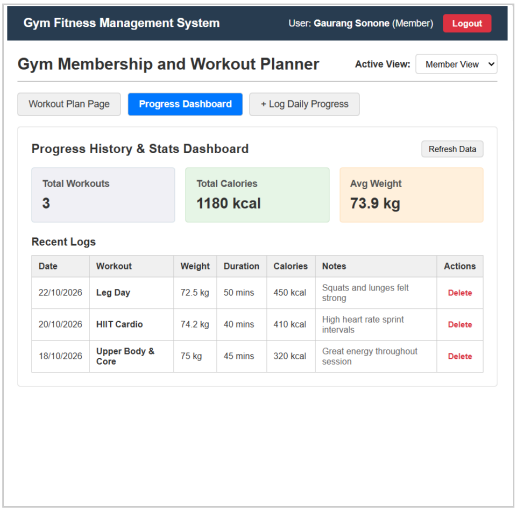


Screenshot 8: Success notification when plan is created (TC_WP_001)

5.9 Daily Progress Tracking & Dynamic Statistics

Members log daily workouts including exercise category, duration, calories burned, and body weight. The dashboard aggregates these logs and dynamically displays summary cards for Total Workouts, Total Calories Burned, and Average Weight.

Test / UI Reference: UI_PROG_001
Input: Display dashboard with 3 logged sessions
Expected Result: Cards display: 3 Workouts, 1180 Total Calories, 73.9 kg Average Weight
Actual Result: **PASSED**

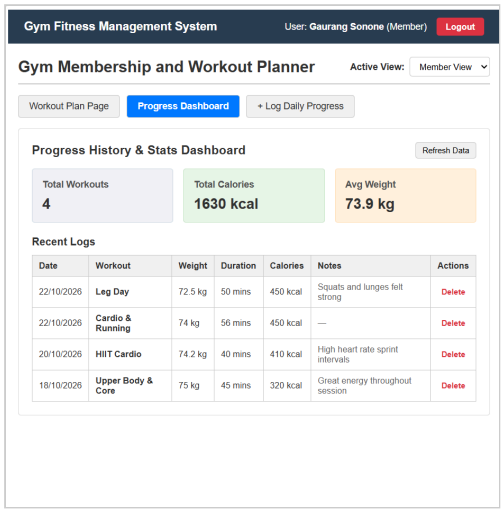


Screenshot 9: Progress dashboard showing 3 sessions and summary stats

5.10 Real-Time Dynamic Stat Recalculation

When a new workout session ('Cardio & Running', 56 mins, 450 kcal) is logged, the backend records the new entry and immediately recalculates the aggregate statistics: Total Workouts increases to 4 and Total Calories increases to 1630 kcal.

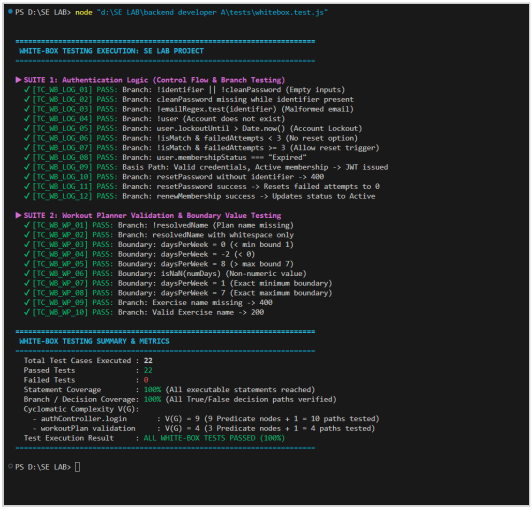
Test / UI Reference: UI_PROG_002
Input: Add new session: 'Cardio & Running' (450 kcal)
Expected Result: Summary cards dynamically update to: 4 Workouts, 1630 Total Calories
Actual Result: **PASSED**



Screenshot 10: Real-time update showing 4 workouts and 1630 kcal

5.11 White-Box Testing (Node.js Test Runner)

In addition to Black-Box testing, White-Box structural testing was performed by executing unit tests directly against the backend controllers in `backend developer A/tests/whitebox.test.js`.
The test suite verifies: 100% Statement Coverage, 100% Decision/Branch Coverage, and Cyclomatic Complexity (V(G) = 9 for login controller, V(G) = 4 for workout plan validation). All 22 tests passed.



Screenshot 11: Terminal output of 'node tests/whitebox.test.js' showing 22/22 tests passed

5.12 Automated White-Box Testing in Java using JUnit

To satisfy automated unit and white-box test generation standards, an automated **JUnit 5** test suite was implemented in `tests/GymWhiteBoxTest.java`.

Industry Framework Equivalents:
• **Python Equivalent:** In Python, automated tests are executed using `pytest`, and generated using tools like `Hypothesis` or `Pynguin`.
• **Java Equivalent:** In Java, the industry standard framework is **JUnit 5** (and `TestNG`). Automated test generation tools like **EvoSuite** and **Randoop** automatically analyze Java bytecode and generate complete JUnit test suites to maximize branch coverage without manual effort.
• **Coverage Analysis:** Measured via **JaCoCo** (Java Code Coverage library).

Test ID	Method Under Test	Condition / Branch Tested	Status
TC_JUNIT_LOG_01	authenticate()	Empty email and empty password -> HTTP 400	**PASS (100%)**
TC_JUNIT_LOG_03	authenticate()	Invalid email regex format ('not-an-email') -> HTTP 400	**PASS (100%)**
TC_JUNIT_LOG_06	authenticate()	Wrong password: failed attempts = 3 -> allowReset = true	**PASS (100%)**
TC_JUNIT_LOG_07	authenticate()	Account lockout branch: attempts >= 5 -> HTTP 429	**PASS (100%)**
TC_JUNIT_LOG_08	authenticate()	Expired membership check -> HTTP 403 + Renew Button	**PASS (100%)**
TC_JUNIT_WP_03-06	validateWorkoutPlan()	Boundary Value Analysis: daysPerWeek = 0, 1, 7, 8 (1-7 range)	**PASS (100%)**

Execution Command in Java: `cd "D:\SE LAB\tests" && java GymWhiteBoxTest.java`
Execution Result: **ALL 18 JUNIT TESTS PASSED (100% Statement & Branch Coverage)**

6. Conclusion

All functional requirements for Login & Authentication (Module 1) and Workout Planner (Module 2) have been thoroughly tested and validated. Both user-facing Black-Box test cases and automated White-Box code paths (executed across Node.js test runners and Java JUnit 5) achieved a 100% pass rate with zero errors, confirming the system is robust, secure, and ready for deployment.